

SmartInterestX

Loan & Interest Management Mobile App

Complete Developer Roadmap

Flutter · Dart · Firebase · Local DB

■ Platform	Android & iOS (Flutter)
■ ■ Stack	Flutter · Dart · Firebase · Hive/SQLite
■ Program	Unlox Academy — Major Project
■ Duration	6–8 Weeks (Build from Scratch)

Step-by-step guide with commands, architecture, code snippets & widget syntax

Table of Contents

1	Project Overview & Architecture	3
2	Environment Setup — Flutter & Firebase CLI	4
3	Flutter Project Structure	5
4	Database Schema & Data Models	6
5	Module 1 — Splash, Onboarding & App Shell	7
6	Module 2 — Contact / Borrower / Lender Management	9
7	Module 3 — Transaction & Interest Calculator	11
8	Module 4 — Payments & Proof Management	13
9	Module 5 — Notifications & Reminders	15
10	Module 6 — Dashboard & Analytics	17
11	Module 7 — Export & Backup	19
12	Firebase Integration (Full Guide)	20
13	State Management with Provider	22
14	Navigation & Routing	23
15	pubspec.yaml — All Dependencies	24
16	Final Submission Checklist	25

1. Project Overview & Architecture

SmartInterestX is a **fintech-style mobile application** built with Flutter and Firebase that helps users manage interest-based transactions between borrowers and lenders. This roadmap is your step-by-step implementation guide — read it top to bottom and build each module one at a time.

1.1 High-Level Architecture Diagram

Layer	Technology	Purpose
UI / Frontend	Flutter (Dart)	Screens, Widgets, Navigation
State Management	Provider	App-wide data reactivity
Local Database	Hive (NoSQL)	Offline-first storage
Cloud Backend	Firebase Firestore	Cloud sync & backup
File Storage	Firebase Storage	Payment proof images
Authentication	Firebase Auth	Google Sign-In / PIN
Notifications	flutter_local_notifications	Scheduled reminders
Export	csv + share_plus	Data export to CSV/Excel
Security	local_auth	Biometric / PIN lock

1.2 App Modules at a Glance

#	Module	Key Features
M1	Splash & Onboarding	Splash screen, intro slides, PIN setup
M2	Contact Management	Add/Edit/Delete borrowers & lenders
M3	Transaction & Interest	Loan entry, SI calculation, due dates
M4	Payments & Proof	Payment recording, receipt upload
M5	Notifications	Scheduled local push notifications
M6	Dashboard & Analytics	Charts, totals, filters
M7	Export & Backup	CSV export, Firebase backup

1.3 Development Phases

Phase 1	Environment Setup Install Flutter, Firebase CLI, create project, connect Firebase	Week 1
Phase 2	Data Layer Define Hive models, Firestore schema, Provider setup	Week 1–2
Phase 3	Core Modules Build M2 + M3 (Contacts + Transactions)	Week 2–4

Phase 4	Payments + Notif Build M4 + M5 (Proof upload + Push notifications)	Week 4–5
Phase 5	Dashboard + Export Build M6 + M7 (Analytics, charts, CSV export)	Week 5–6
Phase 6	Polish & Submit UI polish, testing, APK build, documentation	Week 6–8

2. Environment Setup — Flutter & Firebase CLI

Follow these steps exactly. Every command must run successfully before moving to the next.

2.1 Install Flutter SDK

STEP: Download Flutter SDK from flutter.dev

Go to <https://flutter.dev/docs/get-started/install> and download the SDK zip for your OS. Extract to **C:\flutter** (Windows) or **~/flutter** (Mac/Linux).

STEP: Add Flutter to PATH (Windows)

```
# Add to System Environment Variables → Path:
```

```
$ C:\flutter\bin
```

STEP: Verify Installation

```
$ flutter doctor
```

■ Run flutter doctor and fix ALL issues shown. You need Android Studio or Xcode, Java JDK, and Android SDK.

2.2 Install Android Studio

- Download from developer.android.com/studio
- Install Android SDK, Android Emulator
- In Android Studio → SDK Manager → install API Level 33+
- In Android Studio → AVD Manager → create a virtual device

2.3 Install VS Code (Recommended Editor)

- Download from code.visualstudio.com
- Install Extension: **Flutter** (by Dart Code)
- Install Extension: **Dart** (by Dart Code)
- Install Extension: **Pubspec Assist** (optional but helpful)

2.4 Install Firebase CLI

```
$ npm install -g firebase-tools
```

```
$ firebase login
```

```
$ firebase --version
```

■ You need Node.js installed first. Download from nodejs.org

2.5 Install FlutterFire CLI

```
$ dart pub global activate flutterfire_cli
```

2.6 Create the Flutter Project

STEP: Open terminal in your projects folder and run:

```
$ flutter create smart_interest_x
```

```
$ cd smart_interest_x
```

```
$ flutter run
```

■ Your first app should now run in the emulator. You will see 'Flutter Demo Home Page'. This confirms everything works!

3. Flutter Project Structure

Organize your project this way from the very beginning. A clean folder structure prevents confusion as your app grows.

```
smart_interest_x/<br/>■■■ lib/<br/>■    ■■■ main.dart                ← App entry point<br/>■
■■■ app.dart                ← MaterialApp + Theme<br/>■    ■■■ models/<br/>■    ■    ■■■ cont
act_model.dart<br/>■    ■    ■■■ transaction_model.dart<br/>■    ■    ■■■ payment_model.dart<br
/>■    ■■■ services/<br/>■    ■    ■■■ hive_service.dart ← Local DB operations<br/>■    ■    ■
■■■ firebase_service.dart<br/>■    ■    ■■■ interest_service.dart<br/>■    ■    ■■■ notification
_service.dart<br/>■    ■    ■■■ export_service.dart<br/>■    ■■■ providers/<br/>■    ■    ■■■ c
ontact_provider.dart<br/>■    ■    ■■■ transaction_provider.dart<br/>■    ■    ■■■ auth_provide
r.dart<br/>■    ■■■ screens/<br/>■    ■    ■■■ splash/<br/>■    ■    ■■■ splash_screen.dart
<br/>■    ■    ■■■ onboarding/<br/>■    ■    ■■■ onboarding_screen.dart<br/>■    ■    ■■■ ho
me/<br/>■    ■    ■■■ home_screen.dart<br/>■    ■    ■■■ contacts/<br/>■    ■    ■■■ co
ntact_list_screen.dart<br/>■    ■    ■■■ add_contact_screen.dart<br/>■    ■    ■■■ transact
ions/<br/>■    ■    ■■■ transaction_list_screen.dart<br/>■    ■    ■■■ add_transaction_
screen.dart<br/>■    ■    ■■■ transaction_detail_screen.dart<br/>■    ■    ■■■ payments/<br
/>■    ■    ■■■ payment_screen.dart<br/>■    ■    ■■■ dashboard/<br/>■    ■    ■■■ dash
board_screen.dart<br/>■    ■    ■■■ settings/<br/>■    ■    ■■■ settings_screen.dart<br/>■
■■■ widgets/<br/>■    ■    ■■■ summary_card.dart<br/>■    ■    ■■■ transaction_tile.dart<br/>
■    ■    ■■■ interest_chip.dart<br/>■    ■■■ utils/<br/>■    ■■■ constants.dart<br/>■
■■■ formatters.dart<br/>■    ■■■ validators.dart<br/>■■■ assets/<br/>■    ■■■ images/
<br/>■    ■■■ icons/<br/>■■■ pubspec.yaml                ← All dependencies here<br/>■■■ goog
le-services.json    ← Firebase config (Android)
```

3.1 Create the Folder Structure

In VS Code Terminal, run these commands inside your project:

```
$ mkdir -p lib/models lib/services lib/providers lib/screens/splash
lib/screens/onboarding lib/screens/home lib/screens/contacts lib/screens/transactions
lib/screens/payments lib/screens/dashboard lib/screens/settings lib/widgets lib/utils
assets/images assets/icons
```

■ On Windows PowerShell, use 'md' or create folders manually in Explorer if mkdir -p doesn't work.

4. Database Schema & Data Models

We use **Hive** for local storage (offline-first) and **Firestore** for cloud backup. Define your data models first — everything else depends on them.

4.1 ER Diagram (Entity Relationship)

Entity	Fields	Relationship
Contact	id, name, mobile, email, type (borrower/lender), createdAt	One Contact → Many Transactions
Transaction	id, contactId, amount, type (given/taken), interestRate, startDate, dueDate, notes, status	One Transaction → Many Payments
Payment	id, transactionId, date, amount, mode, proofUrl	Payment belongs to Transaction

4.2 Hive Data Model — Contact

```
// lib/models/contact_model.dart
import 'package:hive/hive.dart';
part 'contact_model.g.dart'; // auto-generated
@HiveType(typeId: 0)
class ContactModel extends HiveObject {
  @HiveField(0) String id;
  @HiveField(1) String name;
  @HiveField(2) String mobile;
  @HiveField(3) String? email;
  @HiveField(4) String type; // 'borrower' or 'lender'
  @HiveField(5) DateTime createdAt;
  ContactModel({
    required this.id,
    required this.name,
    required this.mobile,
    this.email,
    required this.type,
    required this.createdAt,
  });
}
```

4.3 Hive Data Model — Transaction

```
// lib/models/transaction_model.dart
@HiveType(typeId: 1)
class TransactionModel extends HiveObject {
  @HiveField(0) String id;
  @HiveField(1) String contactId;
  @HiveField(2) double amount; // Principal
  @HiveField(3) String type; // 'given' or 'taken'
  @HiveField(4) double interestRate; // Annual rate in %
  @HiveField(5) DateTime startDate;
  @HiveField(6) DateTime? dueDate;
  @HiveField(7) String? notes;
  @HiveField(8) String status; // 'active' | 'partial' | 'settled'
  @HiveField(9) double paidAmount; // Track how much is paid
  TransactionModel({...});
}
```

4.4 Generate Hive Adapters

After defining all model files, run this ONE command to auto-generate adapter files:

```
$ flutter pub run build_runner build --delete-conflicting-outputs
```

■ This generates .g.dart files (e.g. contact_model.g.dart). Never edit these manually.

4.5 Register Hive Adapters in main.dart

```
// lib/main.dart
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  await Hive.initFlutter();
  Hive.registerAdapter(ContactModelAdapter());
  Hive.registerAdapter(TransactionModelAdapter());
  Hive.registerAdapter(PaymentModelAdapter());
  await Hive.openBox<ContactModel>('contacts');
  await Hive.openBox<TransactionModel>('transactions');
  await Hive.openBox<PaymentModel>('payments');
  runApp(const MyApp());
}
```

5. Module 1 — Splash, Onboarding & App Shell

This is the FIRST thing users see. A polished splash + onboarding creates a great first impression.

5.1 Splash Screen

■ Widget: AnimatedContainer + Timer.delayed()

```
// lib/screens/splash/splash_screen.dart<br/>class SplashScreen extends StatefulWidget {<br/>  const SplashScreen({super.key});<br/>  @override State<SplashScreen> createState() => _Splash<br/>ScreenState();<br/>}<br/><br/>class _SplashScreenState extends State<SplashScreen><br/>  with SingleTickerProviderStateMixin {<br/>  late AnimationController _ctrl;<br/>  late Animation<br/>double> _fadeAnim;<br/><br/>  @override<br/>  void initState() {<br/>    super.initState();<br/>    _ctrl = AnimationController(vsync: this,<br/>                                duration: const Duration(millisecon<br/>ds: 1200));<br/>    _fadeAnim = Tween<double>(begin: 0, end: 1).animate(_ctrl);<br/>    _ctrl.<br/>forward();<br/>    // Navigate after 2.5 seconds<br/>    Future.delayed(const Duration(millise<br/>conds: 2500), () {<br/>      Navigator.pushReplacementNamed(context, '/onboarding');<br/>    }<br/>);<br/>  }<br/><br/>  @override<br/>  Widget build(BuildContext context) {<br/>    return Scaf<br/>fold(<br/>      backgroundColor: const Color(0xFF0D1117),<br/>      body: Center(<br/>child: FadeTransition(<br/>        opacity: _fadeAnim,<br/>        child: Column(<br/>          mainAxisAlignment: MainAxisAlignment.center,<br/>          children: [<br/>            Image.asset('assets/images/logo.png', width: 100),<br/>            const SizedBox(heigh<br/>t: 20),<br/>            const Text('SmartInterestX',<br/>              fontSize: 28,<br/>              fontWeight: FontWeight.bold,<br/>              color: Co<br/>lors.white)),<br/>            const SizedBox(height: 8),<br/>            const Text('Loan<br/>& Interest Manager',<br/>              style: TextStyle(color: Colors.blue, fontSize: 14)),<br/>          ],<br/>        ),<br/>      ),<br/>    );<br/>  }<br/>}<br/>}
```

5.2 Bottom Navigation Shell (Home Shell)

■ Widget: Scaffold + BottomNavigationBar

```
// lib/screens/home/home_screen.dart<br/>class HomeScreen extends StatefulWidget {<br/>  const HomeScreen({super.key});<br/>  @override State<HomeScreen> createState() => _HomeScreenState(<br/>);<br/>}<br/><br/>class _HomeScreenState extends State<HomeScreen> {<br/>  int _currentIndex =<br/>0;<br/><br/>  final List<Widget> _pages = [<br/>    DashboardScreen(),<br/>    TransactionLis<br/>tScreen(),<br/>    ContactListScreen(),<br/>    SettingsScreen(),<br/>  ];<br/><br/>  @overrid<br/>e<br/>  Widget build(BuildContext context) {<br/>    return Scaffold(<br/>      body: _pages[_<br/>currentIndex],<br/>      bottomNavigationBar: BottomNavigationBar(<br/>        currentIndex: _<br/>currentIndex,<br/>        onTap: (i) => setState(() => _currentIndex = i),<br/>        selecte<br/>dItemColor: Colors.blue,<br/>        unselectedItemColor: Colors.grey,<br/>        type: Botto<br/>mNavigationBarType.fixed,<br/>        items: const [<br/>          BottomNavigationBarItem(<br/>            icon: Icon(Icons.dashboard), label: 'Dashboard'),<br/>          BottomNavigati<br/>onBarItem(<br/>            icon: Icon(Icons.receipt_long), label: 'Loans'),<br/>          Bo<br/>ttomNavigationBarItem(<br/>            icon: Icon(Icons.people), label: 'Contacts'),<br/>          Bot<br/>tomNavigationBarItem(<br/>            icon: Icon(Icons.settings), label: 'Settings<br/>'),<br/>        ],<br/>      ),<br/>    );<br/>  }<br/>}<br/>}
```

6. Module 2 — Contact / Borrower / Lender Management

Contacts are the backbone of the app. Every loan/transaction is linked to a contact. Build this module before transactions.

6.1 Contact Provider (State Management)

```
// lib/providers/contact_provider.dart
import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';

class ContactProvider extends ChangeNotifier {
  final Box<ContactModel> _box = Hive.box<ContactModel>('contacts');

  List<ContactModel> get allContacts => _box.values.toList();

  List<ContactModel> get borrowers => _box.values.where((c) => c.type == 'borrower').toList();

  List<ContactModel> get lenders => _box.values.where((c) => c.type == 'lender').toList();

  void addContact(ContactModel contact) {
    _box.put(contact.id, contact);
    notifyListeners();
  }

  void updateContact(ContactModel contact) {
    contact.save();
    notifyListeners();
  }

  void deleteContact(String id) {
    _box.delete(id);
    notifyListeners();
  }

  List<ContactModel> search(String query) {
    return _box.values.where((c) => c.name.toLowerCase().contains(query.toLowerCase())).toList();
  }
}
```

6.2 Add Contact Screen

■ Widget: Form + TextFormField + DropdownButtonFormField

```
// Key UI widgets for Add Contact Screen:

// Name field
TextFormField(
  controller: _nameCtrl,
  decoration: InputDecoration(
    labelText: 'Full Name',
    prefixIcon: Icon(Icons.person),
    border: OutlineInputBorder(),
  ),
  validator: (v) => v!.isEmpty ? 'Name is required' : null,
)

// Type picker (Borrower / Lender)
DropdownButtonFormField<String>(
  value: _selectedType,
  decoration: InputDecoration(
    labelText: 'Type',
    border: OutlineInputBorder(),
  ),
  items: ['borrower', 'lender'].map((t) => DropdownMenuItem(value: t, child: Text(t.toUpperCase()))).toList(),
  onChanged: (v) => setState(() => _selectedType = v!),
)

// Save Button
ElevatedButton.icon(
  onPressed: _saveContact,
  icon: Icon(Icons.save),
  label: Text('Save Contact'),
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.blue,
    minimumSize: Size(double.infinity, 48),
  ),
)
```

6.3 Contact List Screen with Search

■ Widget: ListView.builder + SearchBar + Dismissible

```
// lib/screens/contacts/contact_list_screen.dart

// Search bar at top
TextField(
  onChanged: (q) => setState(() => _query = q),
  decoration: InputDecoration(
    hintText: 'Search contacts...',
    prefixIcon: Icon(Icons.search),
    filled: true,
    border: OutlineInputBorder(borderRadius: BorderRadius.circular(12)),
  ),
)

// Filterable list
Consumer<ContactProvider>(
  builder: (context, provider, _) {
    final contacts = _query.isEmpty ? provider.allContacts : provider.search(_query);
    return ListView.builder(
      itemCount: contacts.length,
      itemBuilder: (ctx, i) {
        final c = contacts[i];
        return Dismissible(
          key: Key(c.id),
          direction: DismissDirection.endToStart,
          background: Container(
            color: Colors.red,
            alignment: Alignment.centerRight,
            padding: EdgeInsets.only(right: 16),
            child: Icon(Icons.delete, color: Colors.white),
          ),
          onDismissed: (
```

```

_) => provider.deleteContact(c.id),<br/>          child: ListTile(<br/>            leading: Ci
ircleAvatar(child: Text(c.name[0].toUpperCase()),<br/>              title: Text(c.name),<br/>
            subtitle: Text(c.mobile),<br/>              trailing: Chip(<br/>                label: Te
xt(c.type.toUpperCase()),<br/>              backgroundColor: c.type == 'borrower'<br/>
                ? Colors.orange[100] : Colors.green[100],<br/>              ),<br/>            ),<br/>
);<br/>      },<br/>    );<br/>  },<br/>);

```

7. Module 3 — Transaction & Interest Calculator

This is the **core business logic** of the app. The Interest Calculation Service must be perfectly accurate. Keep it in a separate service class.

7.1 Interest Calculation Service

```
// lib/services/interest_service.dart
class InterestService {
  /// Simple Interest: S
  I = (P × R × T) / 100
  /// P = Principal, R = Annual Rate %, T = Time in years
  static double calculateSI({
    required double principal,
    required double annualRate,
    required DateTime startDate,
    DateTime? endDate,
  }) {
    endDate ??= DateTime.now();
    // Calculate T in years (fractional)
    final days = endDate.difference(startDate).inDays;
    final T = days / 365.0;
    return (principal *
      annualRate * T) / 100;
  }

  /// Monthly Simple Interest
  static double calculateMonthlySI({
    required double principal,
    required double monthlyRate,
    required DateTime startDate,
    DateTime? endDate,
  }) {
    endDate ??= DateTime.now();
    final months = monthsBetween(startDate, endDate);
    return (principal *
      monthlyRate * months) / 100;
  }

  static double totalPayable(double principal, double interest) =>
    principal + interest;

  static int monthsBetween(DateTime from, DateTime to) {
    return (to.year - from.year) * 12 + (to.month - from.month);
  }
}
```

■ **FORMULA:** $SI = (P \times R \times T) / 100$ — This is the key formula. For monthly: divide annual rate by 12 to get monthly rate.

7.2 Add Transaction Screen — Key Form Fields

■ Widget: Form + DatePicker + Real-time interest preview

```
// Amount input
TextFormField(
  controller: _amountCtrl,
  keyboardType: TextInputType.numberWithOptions(decimal: true),
  decoration: InputDecoration(
    labelText: 'Principal Amount (₹)',
    prefixText: '₹ ',
    prefixIcon: Icon(Icons.currency_rupee),
    border: OutlineInputBorder(),
  ),
  onChanged: (_) => _recalculate(), // live preview
  validator: (v) => v!.isEmpty ? 'Amount required' : null,
)

// Given / Taken toggle
SegmentedButton<String>(
  segments: const [
    ButtonSegment(value: 'given', label: Text('Given (I Lent)'), icon: Icon(Icons.arrow_upward)),
    ButtonSegment(value: 'taken', label: Text('Taken (I Borrowed)'), icon: Icon(Icons.arrow_downward)),
  ],
  selected: _type,
  onSelectionChanged: (s) => setState(() => _type = s.first),
)

// Interest Rate
TextFormField(
  controller: _rateCtrl,
  keyboardType: TextInputType.number,
  decoration: InputDecoration(
    labelText: 'Interest Rate (%)',
    suffixText: '% per year',
    prefixIcon: Icon(Icons.percent),
    border: OutlineInputBorder(),
  ),
  onChanged: (_) => _recalculate(),
)

// Date Picker for Start Date
InkWell(
  onTap: () async {
    final picked = await showDatePicker(
      context: context,
      initialDate: _startDate,
      firstDate: DateTime(2000),
      lastDate: DateTime(2100),
    );
    if (picked != null) setState(() { _startDate = picked; _recalculate(); });
  },
  child: InputDecorator(
    decoration: InputDecoration(
      labelText: 'Start Date',
      prefixIcon: Icon(Icons.calendar_today),
      border: OutlineInputBorder(),
    ),
    child: Text(DateFormat('dd MMM yyyy').format(_startDate)),
  ),
)

// Live Interest Preview Card
Card(
  color: Colors.blue[50],
  child: Padding(
    padding: EdgeInsets.all(12),
    child: Column(
      children: [
        _previewRow('Interest Till Today', '₹ ${_interestToday.toStringAsFixed(2)}'),
        _previewRow('Interest Till Due Date', '₹ ${_interestDue.toStringAsFixed(2)}'),
        _previewRow('Total Payable', '₹ ${_totalPayable.toStringAsFixed(2)}', isHighlight: true),
      ],
    ),
  ),
)
```

7.3 Transaction Detail Screen

■ Widget: Card + LinearProgressIndicator (payment progress)

```
// Payment progress indicator<br>Column(<br>  crossAxisAlignment: CrossAxisAlignment.start,<br>  children: [<br>    Text('Payment Progress', style: TextStyle(fontWeight: FontWeight.bold)),<br>    SizedBox(height: 6),<br>    LinearProgressIndicator(<br>      value: transaction.paidAmount / totalPayable,<br>      backgroundColor: Colors.grey[200],<br>      valueColor: AlwaysStoppedAnimation<Color>(Colors.green),<br>      minHeight: 10,<br>    ),<br>    SizedBox(height: 4),<br>    Text('■${transaction.paidAmount} paid of ■$totalPayable'),<br>  ],<br>)</pre>
```


9. Module 5 — Notifications & Reminders

Local notifications remind users about upcoming due dates and overdue payments. We use `flutter_local_notifications` for scheduled local alerts.

9.1 Install Package

```
$ flutter pub add flutter_local_notifications timezone
```

9.2 Notification Service

■ Widget: FlutterLocalNotificationsPlugin

```
// lib/services/notification_service.dart<br>import 'package:flutter_local_notifications/flutter_local_notifications.dart';<br>import 'package:timezone/timezone.dart' as tz;<br>import 'package:timezone/data/latest.dart' as tzd;<br><br>class NotificationService {<br>  static final _plugin = FlutterLocalNotificationsPlugin();<br>  static Future<void> init() async {<br>    tzd.initializeTimeZones();<br>    const android = AndroidInitializationSettings('@mipmap/ic_launcher');<br>    const ios = DarwinInitializationSettings();<br>    await _plugin.initialize(<br>      const InitializationSettings(android: android, iOS: ios),<br>    );<br>  }<br><br>  // Schedule a notification for a due date<br>  static Future<void> scheduleDueReminder({<br>    required int id,<br>    required String title,<br>    required String body,<br>    required DateTime scheduledDate,<br>    int daysBefore = 1,<br>  }) async {<br>    final notifyAt = scheduledDate.subtract(Duration(days: daysBefore));<br>    if (notifyAt.isBefore(DateTime.now())) return; // skip past dates<br>    await _plugin.zonedSchedule(<br>      id,<br>      title,<br>      body,<br>      tz.TZDateTime.from(notifyAt, tz.local),<br>      const NotificationDetails(<br>        android: AndroidNotificationDetails(<br>          'due_channel', 'Due Date Reminders',<br>          importance: Importance.high,<br>          priority: Priority.high,<br>          icon: '@mipmap/ic_launcher',<br>        ),<br>      uiLocalNotificationDateInterpretation: UILocalNotificationDateInterpretation.absoluteTime,<br>      androidScheduleMode: AndroidScheduleMode.exactAllowWhileIdle,<br>    );<br>  }<br><br>  // Show instant notification<br>  static Future<void> showNow({<br>    required int id,<br>    required String title,<br>    required String body,<br>  }) async {<br>    await _plugin.show(id, title, body,<br>      const NotificationDetails(<br>        android: AndroidNotificationDetails(<br>          'instant_channel', 'Instant Alerts',<br>          importance: Importance.defaultImportance,<br>        ),<br>    );<br>  }<br><br>  static Future<void> cancelAll() => _plugin.cancelAll();<br>}<br>
```

9.3 Schedule Reminder When Transaction is Added

```
// After saving a transaction, schedule its reminder:<br>if (transaction.dueDate != null) {<br>  // 1 day before due<br>  await NotificationService.scheduleDueReminder(<br>    id: transaction.id.hashCode,<br>    title: '■ Payment Due Soon!,<br>    body: '${contact.name} - ■${transaction.amount} due tomorrow',<br>    scheduledDate: transaction.dueDate!,<br>    daysBefore: 1,<br>  );<br>  // 3 days before due<br>  await NotificationService.scheduleDueReminder(<br>    id: transaction.id.hashCode + 1,<br>    title: '■■ Upcoming Due Date',<br>    body: '${contact.name} - ■${transaction.amount} due in 3 days',<br>    scheduledDate: transaction.dueDate!,<br>    daysBefore: 3,<br>  );<br>}<br>
```

■ Add `SCHEDULE_EXACT_ALARM` permission in `AndroidManifest.xml` for Android 12+

```
<uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM"/><br><uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
```

9.4 Settings Screen — Reminder Preference

■ Widget: RadioListTile for reminder days selection

```
// Reminder days selector<br>int _reminderDays = 1;<br>final options = [1, 3, 7];<br><br>C<br>olumn(<br>  children: options.map((day) => RadioListTile<int>(<br>    title: Text('$day day$<br>{day > 1 ? 's' : ''} before due date'),<br>    value: day,<br>    groupValue: _reminderDays,<br>    activeColor: Colors.blue,<br>    onChanged: (v) => setState(() => _reminderDays = v!<br>),<br>  )).toList(),<br>)
```

10. Module 6 — Dashboard & Analytics

The dashboard is the FIRST screen users see after login. It must show a powerful overview with summary cards and charts.

10.1 Install Chart Package

```
$ flutter pub add fl_chart
```

10.2 Summary Cards

■ Widget: GridView.count + Custom SummaryCard widget

```
// lib/widgets/summary_card.dart
class SummaryCard extends StatelessWidget {
  final String title;
  final String amount;
  final IconData icon;
  final Color color;

  SummaryCard({
    super.key,
    required this.title,
    required this.amount,
    required this.icon,
    required this.color,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 3,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12)),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Icon(icon, color: color, size: 28),
            const SizedBox(height: 8),
            Text(amount, style: TextStyle(
              fontSize: 20,
              fontWeight: FontWeight.bold,
              color: color)),
            const SizedBox(height: 4),
            Text(title, style: TextStyle(
              fontSize: 12,
              color: Colors.grey[600])),
          ],
        ),
      ),
    );
  }
}

// Usage in DashboardScreen:
GridView.count(
  crossAxisCount: 2,
  crossAxisSpacing: 12,
  mainAxisSpacing: 12,
  shrinkWrap: true,
  physics: NeverScrollableScrollPhysics(),
  children: [
    SummaryCard(
      title: 'Interest Earned',
      amount: '■$[_totalEarned.toStringAsFixed(0)]',
      icon: Icons.trending_up,
      color: Colors.green),
    SummaryCard(
      title: 'Interest Paid',
      amount: '■$[_totalPaid.toStringAsFixed(0)]',
      icon: Icons.trending_down,
      color: Colors.red),
    SummaryCard(
      title: 'Receivables',
      amount: '■$[_receivables.toStringAsFixed(0)]',
      icon: Icons.account_balance_wallet,
      color: Colors.blue),
    SummaryCard(
      title: 'Payables',
      amount: '■$[_payables.toStringAsFixed(0)]',
      icon: Icons.payment,
      color: Colors.orange),
  ],
)
```

10.3 Pie Chart — Given vs Taken

■ Widget: fl_chart PieChart

```
// lib/screens/dashboard/dashboard_screen.dart
PieChart(
  PieChartData(
    sections: [
      PieChartSectionData(
        value: _givenTotal,
        title: 'Given\n■$[_givenTotal.toInt()]',
        color: Colors.green,
        radius: 80,
        titleStyle: TextStyle(
          color: Colors.white,
          fontWeight: FontWeight.bold,
          fontSize: 12),
      ),
      PieChartSectionData(
        value: _takenTotal,
        title: 'Taken\n■$[_takenTotal.toInt()]',
        color: Colors.orange,
        radius: 80,
        titleStyle: TextStyle(
          color: Colors.white,
          fontWeight: FontWeight.bold,
          fontSize: 12),
      ),
    ],
    sectionsSpace: 4,
    centerSpaceRadius: 40,
  ),
)
```

10.4 Bar Chart — Monthly Interest Flow

■ Widget: fl_chart BarChart

```
BarChart(  
  BarChartData(  
    barGroups: _monthlyData.asMap().entries.map((e) =>  
      BarChartGroupData(x: e.key, barRods: [  
        BarChartRodData(  
          value.earned,<br/>          color: Colors.green,<br/>          width: 12,<br/>        ),<br/>  
        BarChartRodData(  
          toY: e.value.paid,<br/>          color: Colors.red,<br/>          width: 12,<br/>        ),<br/>      ])<br/>    ).toList(),<br/>    titlesData: FlTitlesData(  
      bottomTitles: AxisTitles(  
        showTitles: true,<br/>        getTitlesWidget: (v, meta) =><br/>          Text(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',<br/>            'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'][v.toInt()],<br/>            style: TextStyle(fontSize: 9)),<br/>      ),<br/>    ),<br/>  ),<br/>)
```

11. Module 7 — Export & Backup

Let users export their data as CSV and backup to Firebase. This builds trust and is a key differentiator.

11.1 Install Packages

```
$ flutter pub add csv path_provider share_plus
```

11.2 Export to CSV

■ Widget: File + CsvToListConverter + Share

```
// lib/services/export_service.dart
import 'package:csv/csv.dart';
import 'package:path_provider/path_provider.dart';
import 'package:share_plus/share_plus.dart';
import 'dart:io';

class ExportService {
  static Future<void> exportTransactionsToCSV(
    List<TransactionModel> transactions,
    Map<String, ContactModel> contactsMap,
  ) async {
    // Build CSV rows
    final List<List<dynamic>> rows = [
      // Header row
      ['ID', 'Contact Name', 'Type', 'Amount (Rs)', 'Interest Rate (%)', 'Start Date', 'Due Date', 'Interest Earned', 'Total Payable', 'Status'],
    ];
    for (final txn in transactions) {
      final contact = contactsMap[txn.contactId];
      final interest = InterestService.calculateSI(
        principal: txn.amount,
        annualRate: txn.interestRate,
        startDate: txn.startDate,
        endDate: txn.dueDate,
      );
      rows.add([
        txn.id, contact?.name ?? 'Unknown',
        txn.type, txn.amount, txn.interestRate,
        DateFormat('dd-MM-yyyy').format(txn.startDate),
        txn.dueDate != null ? DateFormat('dd-MM-yyyy').format(txn.dueDate!) : 'No due date',
        interest.toStringAsFixed(2),
        (txn.amount + interest).toStringAsFixed(2),
        txn.status,
      ]);
    }
    final csv = const ListToCsvConverter().convert(rows);
    final dir = await getApplicationDocumentsDirectory();
    final filePath = '${dir.path}/smartinterestx_export.csv';
    final file = File(filePath);
    await file.writeAsString(csv);
    // Share the file
    await Share.shareXFiles([XFile(filePath)], text: 'SmartInterestX - Transaction Export');
  }
}
```

11.3 Cloud Backup to Firebase Firestore

■ Widget: FirebaseFirestore batch write

```
// lib/services/firebase_service.dart
Future<void> backupContactsToFirestore(
  String userId, List<ContactModel> contacts,
) async {
  final db = FirebaseFirestore.instance;
  final batch = db.batch();
  for (final c in contacts) {
    final ref = db.collection('users/$userId/contacts').doc(c.id);
    batch.set(ref, {
      'name': c.name, 'mobile': c.mobile, 'email': c.email, 'type': c.type, 'createdAt': c.createdAt.toIso8601String(),
    });
  }
  await batch.commit();
}
```

12. Firebase Integration — Full Guide

Follow this exact sequence. Missing any step will cause your app to crash.

Step 1 — Create Firebase Project

- Go to console.firebase.google.com
- Click **Add Project** → Name it: SmartInterestX
- Disable Google Analytics (optional for now) → Click Create

Step 2 — Connect Firebase to Flutter App

```
$ firebase login
```

```
$ dart pub global activate flutterfire_cli
```

```
$ flutterfire configure
```

■ Select your Firebase project when prompted. This auto-generates `lib/firebase_options.dart`

Step 3 — Add Firebase Packages to pubspec.yaml

```
$ flutter pub add firebase_core firebase_auth cloud_firestore firebase_storage
```

Step 4 — Initialize Firebase in main.dart

```
// lib/main.dart<br>import 'package:firebase_core/firebase_core.dart';<br>import 'firebase_options.dart';<br><br>void main() async {<br>  WidgetsFlutterBinding.ensureInitialized();<br>  await Firebase.initializeApp(<br>    options: DefaultFirebaseOptions.currentPlatform,<br>  );<br>  // ... rest of init<br>  runApp(const MyApp());<br>}
```

Step 5 — Firestore Security Rules

In Firebase Console → Firestore → Rules, paste this:

```
rules_version = '2';<br>service cloud.firestore {<br>  match /databases/{database}/documents<br>  {<br>    match /users/{userId}/{document=**} {<br>      allow read, write: if request.auth<br>      != null<br>      && request.auth.uid == userId;<br>    }<br>  }<br>}
```

Step 6 — Google Sign-In Setup (Optional)

```
$ flutter pub add google_sign_in
```

```
// lib/services/auth_service.dart<br>import 'package:firebase_auth/firebase_auth.dart';<br>import 'package:google_sign_in/google_sign_in.dart';<br><br>class AuthService {<br>  final _auth = FirebaseAuth.instance;<br>  final _google = GoogleSignIn();<br><br>  Future<UserCredential?> signInWithGoogle() async {<br>    try {<br>      final googleUser = await _google.signIn();<br>      if (googleUser == null) return null;<br>      final googleAuth = await googleUser.authentication;<br>      final credential = GoogleAuthProvider.credential(<br>        accessToken: googleAuth.accessToken,<br>        idToken: googleAuth.idToken,<br>      );<br>      return await _auth.signInWithCredential(credential);<br>    } catch (e) {<br>      debugPrint('Auth error: $e');<br>      return null;<br>    }<br>  }<br><br>  Future<void>
```

```
signOut() async {  
  await _google.signOut();  
  await _auth.signOut();  
}  
User? get currentUser => _auth.currentUser;  
Stream<User?> get authStateChanges =  
  _auth.authStateChanges();
```

13. State Management with Provider

Provider is the recommended state management solution for this app. It connects your data layer (Hive/Firebase) to the UI layer (Screens).

13.1 Install Provider

```
$ flutter pub add provider
```

13.2 Register Providers in app.dart

```
// lib/app.dart
import 'package:provider/provider.dart';

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => ContactProvider()),
        ChangeNotifierProvider(create: (_) => TransactionProvider()),
        ChangeNotifierProvider(create: (_) => AuthProvider()),
      ],
      child: MaterialApp(
        title: 'SmartInterestX',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
          useMaterial3: true,
          fontFamily: 'Roboto',
        ),
        initialRoute: '/splash',
        routes: {
          '/splash': (ctx) => const SplashScreen(),
          '/onboarding': (ctx) => const OnboardingScreen(),
          '/home': (ctx) => const HomeScreen(),
          '/add-contact': (ctx) => const AddContactScreen(),
          '/add-txn': (ctx) => const AddTransactionScreen(),
        },
      ),
    );
  }
}
```

13.3 Consuming Provider in Widgets

■ Widget: Consumer and context.watch()

```
// Method 1: Consumer widget (recommended for rebuilding only part of UI)
Consumer<TransactionProvider>(
  builder: (context, provider, child) {
    return ListView.builder(
      itemCount: provider.activeTransactions.length,
      itemBuilder: (ctx, i) =>
        TransactionTile(transaction: provider.activeTransactions[i]),
    );
  },
)

// Method 2: context.read (for actions, does NOT rebuild)
ElevatedButton(
  onPressed: () {
    context.read<ContactProvider>().addContact(newContact);
    Navigator.pop(context);
  },
  child: Text('Save'),
)

// Method 3: context.watch (rebuilds when data changes)
final contacts = context.watch<ContactProvider>().allContacts;
```

14. Navigation & Routing

Use named routes for clean, maintainable navigation throughout your app.

14.1 Navigate to a Screen

```
// Push to new screen (add to history)<br>Navigator.pushNamed(context, '/add-contact');<br><br>
// Push with arguments (passing data)<br>Navigator.pushNamed(context, '/transaction-detail',<br>  arguments: {'transactionId': txn.id});<br><br>
// Read arguments in destination screen<br>final args = ModalRoute.of(context)!.settings.arguments as Map<String, dynamic>;<br>final txnId = args['transactionId'];<br><br>
// Replace current screen (no back button)<br>Navigator.pushReplacementNamed(context, '/home');<br><br>
// Go back<br>Navigator.pop(context);<br><br>
// Go back with result<br>Navigator.pop(context, true); // returns 'true' to previous screen
```

14.2 User Flow Diagram

The complete user flow of the app:

Step	Screen	Action	Next
1	Splash Screen	App opens	→ Onboarding (first time) / Home
2	Onboarding	View intro slides	→ Home Screen
3	Home / Dashboard	See summary cards & charts	→ Add Loan / Contacts
4	Contacts Screen	Search & tap a contact	→ Add Contact / Txn List
5	Add Transaction	Fill form, see live interest	→ Transaction Detail
6	Transaction Detail	View interest due, history	→ Record Payment
7	Payment Screen	Pay, upload proof	→ Dashboard (updated)
8	Dashboard	View updated analytics	→ Export / Backup
9	Settings	Reminder prefs, backup	→ Home

15. pubspec.yaml — All Dependencies

Copy this complete pubspec.yaml for your project. Run **flutter pub get** after pasting.

```
name: smart_interest_x<br/>description: Loan & Interest Management App<br/>publish_to: 'none'<br/>version: 1.0.0+1<br/><br/>environment:<br/>  sdk: '>=3.0.0 <4.0.0'<br/><br/>dependencies:<br/>  flutter:<br/>    sdk: flutter<br/><br/>  # State Management<br/>  provider: ^6.1.1<br/><br/>  # Local Database<br/>  hive: ^2.2.3<br/>  hive_flutter: ^1.1.0<br/><br/>  # Firebase<br/>  firebase_core: ^2.24.2<br/>  firebase_auth: ^4.16.0<br/>  cloud_firestore: ^4.14.0<br/>  firebase_storage: ^11.6.0<br/>  google_sign_in: ^6.2.1<br/><br/>  # Notifications<br/>  flutter_local_notifications: ^16.3.0<br/>  timezone: ^0.9.2<br/><br/>  # Charts<br/>  fl_chart: ^0.67.0<br/><br/>  # Image<br/>  image_picker: ^1.0.7<br/><br/>  # Export<br/>  csv: ^6.0.0<br/>  path_provider: ^2.1.2<br/>  share_plus: ^7.2.2<br/><br/>  # Date formatting<br/>  intl: ^0.19.0<br/><br/>  # UUID generation<br/>  uuid: ^4.3.3<br/><br/>  # Security<br/>  local_auth: ^2.1.8<br/><br/>  # Utils<br/>  cupertino_icons: ^1.0.6<br/><br/>dev_dependencies:<br/>  flutter_test:<br/>    sdk: flutter<br/>  hive_generator: ^2.0.1<br/>  build_runner: ^2.4.8<br/>  flutter_lints: ^3.0.0<br/><br/>flutter:<br/>  uses-material-design: true<br/>  assets:<br/>    - assets/images/<br/>    - assets/icons/
```

```
$ flutter pub get
```

■ Run flutter pub get every time you edit pubspec.yaml

15.1 Key Commands Reference

Command	Purpose
flutter create smart_interest_x	Create new Flutter project
flutter pub get	Install all dependencies
flutter run	Run app in connected device/emulator
flutter run -d chrome	Run as web app (for testing)
flutter pub run build_runner build	Generate Hive adapter files
flutter build apk --release	Build release APK for Android
flutter build apk --debug	Build debug APK
flutter doctor	Check environment setup
flutterfire configure	Connect Firebase to Flutter
flutter pub add	Add a new package
flutter pub upgrade	Upgrade all packages
flutter clean	Clear build cache (when build fails)
flutter analyze	Check for code errors

16. Final Submission Checklist

Before submitting, verify every item on this checklist. Your mentor will evaluate all of these.

■ Source Code

- Flutter project compiles without errors
- All modules implemented (M1–M7)
- Code organized as per folder structure in Section 3
- No API keys hardcoded (use .env or Firebase configs)
- Meaningful variable and function names used

■ APK Build

- Debug APK generated successfully
- App installs on Android device
- All screens load without crash
- Internet permission added in AndroidManifest

■ Database & Schema

- ER Diagram created (3 entities: Contact, Transaction, Payment)
- Hive models defined with correct @HiveType and @HiveField
- Firestore collection structure documented
- build_runner successfully generates .g.dart files

■ Screenshots

- Splash Screen
- Onboarding Screen
- Dashboard with Charts
- Contact List + Search
- Add Contact Form
- Transaction List
- Add Transaction with live interest preview
- Transaction Detail with payment progress
- Payment Recording + Proof upload
- Notification settings
- Export / Backup screen

■ Notifications

- Local notification fires 1 day before due date
- Local notification fires 3 days before due date

- Notification taps open correct transaction

■ Dashboard

- Total interest earned card visible
- Total interest paid card visible
- Pie chart: Given vs Taken loads with data
- Bar chart: Monthly interest shows correctly
- Filter by month/year/contact works

■ Export

- Export to CSV works and produces valid file
- Share dialog appears after export
- CSV opens in Google Sheets / Excel correctly
- Firebase backup uploads data to Firestore

■ Documentation

- Tech stack list written
- Data model diagram or table included
- Interest calculation formula explained
- Notification scheduling approach described
- Backup & export flow explained

■ Demo Video (3–5 min)

- Add new contact shown
- Add transaction with interest preview shown
- Payment recording with proof upload shown
- Dashboard with charts shown
- Export CSV shown

■ You're Ready to Submit!

Built with ❤️ for Unlox Academy Students | Stack: Flutter · Dart · Firebase · Hive